

Práctica 3: **Algoritmos Voraces**

Realizado por:

Jairo Robles Balderas

Hugo Bautista Arenas

Francisco Domene Luján

Álvaro García Zafra

Diego Cabrero Martinez

Problema 1

1. Diseño de componentes:

- a. Lista de candidatos: todas las posibles parejas.
- b. Lista de candidatos seleccionados: las parejas que se han ido seleccionando.
- c. Función solución: la suma de los emparejamientos seleccionados .
- d. Función de factibilidad: el número de personas ha de ser par para que sea una solución factible.
- e. Función selección: el emparejamiento que mejor puntuación tenga con cada persona.
- f. Función objetivo: La suma de todos los emparejamientos.

2. Diseño del algoritmo

1. Inicialización:

- Se crea un tipo de dato llamado Mejor que se usa para almacenar el entero de la posición con el que mejor combina hasta el momento y almacenar la cantidad de emparejamiento de la pareja
- Se inicializa una semilla de aleatorios para la creación de la matriz
- Se inicializa la matriz con aleatorios salvo la posiciones (i,i) y un vector que será donde se almacenarán las parejas

2. Funciones auxiliares:

- Función esta: devuelve true si el valor está contenido en el vector
- Función imprime: Imprime el vector de las parejas
- Función imprimeMatriz: Imprime la matriz que se le pasa.

3. Algoritmo voraz:

- Se inicializa a 0 ambos valores de un dato Mejor
- Empieza un bucle que recorre las casillas de la matriz. Si la posición actual no está ya almacenada, se almacena y busca su mejor emparejamiento usando el tipo de dato inicializado sin tenerse en cuenta a sí mismo.
- Devuelve la suma total de todos los emparejamientos.

3. Estudio de optimalidad:

Pongamos el caso que se genera la siguiente matriz:

-	25	1	50
100	-	100	1
1	100	-	1
50	1	1	-

En este caso el algoritmo escoge por primera vez la posición 1. Por tanto el algoritmo se decanta por el 2 ya que es el primero en dar 2500 y se quedarían el 3 y

el 4 juntos, dando 1. Por tanto la suma es de 2501. Si ponemos al 2 con el 3 daría 10000 de compatibilidad y el 1 con el 4 tendrías 2500. La suma total es de 12500, por tanto, por contradicción, el algoritmo no es óptima.

4. Ejemplo paso a paso:

-	48	23	90	35	15
56	-	72	33	21	97
96	9	-	12	87	67
76	32	49	-	55	83
34	10	92	56	-	84
69	52	90	2	86	-

Paso 1: Empieza el algoritmo cogiendo la primera fila y comprueba con 2, con 3, con 4, con 5 y con 6. El mayor es el 4 con 6840.

Paso 2: El siguiente sería el 2 ya que no ha sido ocupado antes. Comprueba con el 3, con el 5 y con el 6. El mayor es el 6 con 5044. Suma la cantidad a la anterior.

Paso 3: El siguiente sería el 3 ya que no ha sido usado antes. Comprueba con el 5 que es el único restante y el resultado es de 8004. Devuelve la suma de los 3 que sería 19888.

El código implementado en C++ del algoritmo está en el fichero “1.cpp”

Problema 2

1. Diseño de componentes:

- a. Lista de candidatos: todas las posibles parejas
- b. Lista de candidatos seleccionados: las parejas que se han ido seleccionando
- c. Función solución: la mejor pareja a la izquierda disponible
- d. Función selección: el emparejamiento que mejor puntuación tenga con alguien disponible y en la izquierda

2. Diseño del algoritmo:

1. Inicialización:

- Se inicializa una variable entera a 0, indicando que el primero en sentarse es el invitado 0
- Se inicializa un contador del numero de invitados sentados a 0
- Se inicializa un vector de enteros que va a contener el orden de los invitados, inicializado a 0 y de tamaño 1, lo cual indica que el invitado 0 es el primero en sentarse

2. Funciones auxiliares:

- Función encuentraMaximo: dado un vector de int, devuelve el índice del número mayor

3. Algoritmo voraz:

- Se inicializa a 0 el índice que itera en los invitados que están sentados y buscan pareja a la derecha.
- Empieza un bucle que recorre la fila de dicho invitado, y busca el invitado disponible con mayor conveniencia
- Una vez encontrada la pareja, la matriz es modificada, y en los valores en los que el invitado interviene (osea, su fila y su columna) se iguala a 0, indicando que ya no está disponible para sentarse.
- Se añade al final del vector de orden dicho invitado
- El índice ahora toma el valor de la último invitado seleccionado, el cual vuelve a buscar alguien a su izquierda

3. Estudio de optimalidad:

Pongamos en ejemplo la siguiente matriz:

0	40	90	7
40	0	85	75
90	85	0	80
7	75	80	0

Este algoritmo el problema que tiene es que conforme van quedando menos candidatos posibles, es más probable que alguien tenga a su izquierda a alguien con el cual no tiene buena conveniencia. En este caso, empareja al 1 con el 4, que tienen una conveniencia pésima.

4. Ejemplo paso a paso:

Tomamos a 4 invitados numerados del 0 al 3.

0	40	90	7
40	0	85	75
90	85	0	80
7	75	80	0

Vector de orden:

0			
---	--	--	--

Se sienta al invitado 0 y se le busca pareja para ponerla a la izquierda.

Se escoge al invitado 2, el cuál se sienta y busca pareja:

0	0	0	0
0	0	85	75
0	85	0	80
0	75	80	0

Vector de orden:

2	0		
---	---	--	--

Se escoge al invitado 1, el cuál se sienta y busca pareja:

0	0	0	0
0	0	0	75
0	0	0	0
0	75	0	0

Vector de orden:

2	1	3	0
---	---	---	---

Encuentra al único disponible, el invitado 3.

Para dibujar la mesa de invitados basta con poner primero al último elemento, y después del primero al penúltimo a la izquierda.

0
3 (centro) 2
1

El código implementado en C++ del algoritmo está en el fichero “2.cpp”

Problema 3

1. Diseño de componentes

- a. Lista de candidatos: todas las gasolineras del entorno.
- b. Lista de candidatos usables: las gasolineras a las que puede ir con los kilómetros disponibles.
- c. Función solución: llega al destino esperado.
- d. Función objetivo: llegar al destino haciendo el número mínimo de paradas.
- e. Criterio de selección: ir a la parada que gaste menos kilómetros.
- f. Criterio de factibilidad: va a las gasolineras si tiene kilómetros suficientes.

2. Diseño del algoritmo

1. Inicialización:
Se inicializan las gasolineras y los kilómetros iniciales.
Se inicializan las funciones auxiliares para decidir el siguiente movimiento.
Se crea un booleano que indique que está en el destino deseado
2. Funciones auxiliares:
Se crea la función Decidir_nodo, que decide a qué nodo irse dependiendo de cuál tiene menos kilómetros, dentro de esta función se utilizará también la función de Kilometros_suficientes.
Se crea la función Kilometros_suficientes, que se le pasa el nodo siguiente como parámetro y devuelve true si tiene kilómetros suficientes para llegar a ese nodo.
3. Algoritmo voraz: tendríamos un bucle while en el que itera siempre que no haya llegado al destino, en cada iteración se selecciona el nodo con la función de Decidir_nodo, si se cumple la condición de Kilometros_suficientes, va hacia ese nodo, actualizando la posición y los kilómetros de autonomía.

3. Estudio de optimalidad

El algoritmo propuesto, como podemos ver se basa en elegir la mejor acción a corto plazo, lo cual a la hora de elegir la elección más óptima no siempre es así, ya que podemos poner un ejemplo donde no escogería el camino más óptimo, la situación es la siguiente, estamos en nodo y comprobamos que hay dos caminos disponibles, el primero necesitas 40km y el segundo 30km, por lo cual se iría por el segundo camino, pero justo para llegar al destino final, eligiendo el primer camino hubiera

gastado menos kilómetros en total, pero acaba gastando más por el segundo camino.

En definitiva, el algoritmo voraz es bastante óptimo pero hay ejemplos donde no es el mejor.

4. Ejemplo paso a paso

		30km				
40km				15km		Dest
		60km				

Suponiendo que estamos por ejemplo en el nodo de 40km, vamos a ver cómo llegaríamos al destino pasando por las menos paradas posibles. Sus conexiones posibles son la de 30km y el de 60km, por lo cual, elegiría el camino de 30km, y después tendría la posibilidad de ir al de 60km y al de 15 km se iría al de 15km y ya solo le quedaría elegir entre el Destino, el cual siempre que tiene la posibilidad, se dirigirá hacia él, y 60km, por lo cual llegará al destino exitosamente.

Problema 4

1. Diseño de componentes:

- a. Lista de candidatos: todos los sensores del entorno.
- b. Lista de candidatos seleccionados: la distancia que hay entre los sensores.
- c. Función solución: el número de vértices por los que ha pasado.
- d. Función de selección: se selecciona el nodo adyacente de menor distancia.
- e. Función de factibilidad: una solución es factible si desde cualquier nodo puedes llegar al nodo central.
- f. Función objetivo: Minimizar el tiempo de transmisión desde cada nodo hasta el nodo central.

2. Diseño del algoritmo:

1. Inicialización:

- Se crea un dato Nodo, el cuál se va a usar para representar cada uno de los componentes del grafo.
- Se crea un dato solucion, el cual representa los nodos por lo saque tiene que pasar.

2. Funciones auxiliares:

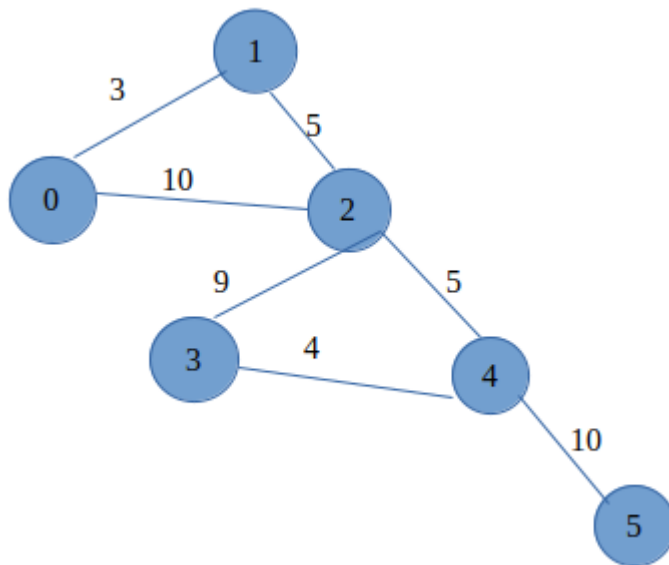
- Se crea una función MenorCoste, la cual nos va a devolver cual es el nodo disponible más cercano.
- Se crea una función Encontrar, la cual devuelve true si encuentra el id de un nodo en un vector de nodos.

3. Algoritmo Dijkstra (si el vector no está vacío)

- Se crea un vector seleccionados y le añadimos al final el nodo central.
- Se crea un while, del que no se saldrá mientras que el número de nodos seleccionados sea menor que el tamaño del grafo.
 - Dentro de este bucle contamos con dos bucles for anidados, el primero recorre el vector seleccionados y el segundo recorre sus nodos adyacentes.
 - Dentro de estos contamos con un if, este comprueba para cada nodo adyacente si ya ha sido seleccionado, si no lo ha sido entra y actualiza la dist del nodo adyacente y verifica si es menor que la variable costmenor, en el caso de que sea así actualiza los datos.
 - Finalmente este nodo se agrega a un vector llamado candidatos.
 - Una vez acabados los bucles anidados actualiza la distancia del nodo seleccionado con la distancia almacenada en el grafo candidato y agrega el id del nodo al camino de la solución.

3. Estudio optimalidad

Este algoritmo se encarga de buscar un óptimo local en lugar de un óptimo global lo cual en algunas situaciones no nos va a dar un resultado minimizado, un ejemplo puede ser el grafo que tenemos a continuación. Este grafo en su recorrido más rápido tardará un tiempo de 23, mientras que recorriéndolo con este algoritmo tardará un tiempo de 33. Como podemos ver este algoritmo no es óptimo a la hora de buscar el recorrido más corto, ya que siempre avanzará a su adyacente más corto sin importar qué habrá tras ese.



4. Ejemplo paso a paso

Vamos a ver el comportamiento del grafo anterior paso a paso:

Inicialización: comenzamos desde el nodo 5 y queremos llegar al nodo 0.

1º solo tenemos un camino así que se mueve hasta el nodo 4.

2º como este nodo tiene 2 adyacentes elige el que tarde menos, el cual es ir hacia el nodo 3.

3º del nodo 3 solo puede ir al nodo 2, por lo que avanza.

4º una vez en este nodo tenemos 2 tenemos dos posibilidades, ir al nodo 1, la cual es la opción más rápida o ir al nodo 0, que es el nodo central, aunque el camino de ir al nodo 1 y después al 0 es más rápido nos dirigimos directamente hacia el 0, ya que con este algoritmo sólo conocemos el tiempo de los nodos que tenemos adyacentes.

Problema 5

1. Diseño de componentes:

- a. Lista de candidatos: Todas las calles que conectan las plazas.
- b. Lista de candidatos usados: Las calles seleccionadas para asfaltar.
- c. Criterio de selección: Seleccionar la calle con el menor costo de asfaltado que no forme un ciclo con las calles ya seleccionadas.
- d. Criterio de factibilidad: Una calle es factible si su inclusión no forma un ciclo.
- e. Función solución: Cuando todas las plazas están conectadas por las calles seleccionadas.
- f. Función objetivo: Minimizar el costo total de asfaltado.

2. Diseño del algoritmo:

1. Inicialización:

- Se crea un vector *padre* y un vector *rango* de tamaño *n* (número de plazas).
- //el vector *rango* se usa para la unión de los conjuntos(las plazas conectadas), el conjunto con más rango será el conjunto padre
- Se crea un vector *calles* para almacenar todas las calles.
- Se llama a la función *crear_conjunto* para cada plaza, que inicializa *padre[i] = i* y *rango[i] = 0*.

2. Agregar calles:

- Se lee el número de calles *m*.
- Para cada calle, se leen *plaza1*, *plaza2* y *costo*, y se llama a la función *agregar_calle* para agregar la calle al vector *calles*.

3. Algoritmo de Kruskal:

- Se crea un vector *usada* de tamaño *m* para llevar un seguimiento de las calles que ya se han usado.
- Para *i* desde 0 hasta *m-1*:
 - Se busca la calle de menor costo que no se ha usado aún. Para ello, se recorren todas las calles y se selecciona la de menor costo que no se ha usado (*usada[j] == false*).
 - Si se encuentra una calle de menor costo (*min_calle != -1*), se comprueba si las plazas que conecta están en el mismo conjunto. Si no lo están (*encontrar_conjunto(plaza1) != encontrar_conjunto(plaza2)*), se imprime la calle y se une los conjuntos de *plaza1* y *plaza2*.
 - Se marca la calle como usada (*usada[min_calle] = true*).

3. Estudio de optimalidad:

En este caso, vamos a considerar n como el número de aristas en el árbol de expansión mínima.

Demostración por inducción:

Paso base: Cuando $n=1$, solo hay una arista en el árbol. El algoritmo de Kruskal selecciona la arista de menor peso, por lo que en este caso, el árbol producido por el algoritmo de Kruskal es óptimo.

Hipótesis de inducción: Supongamos que el algoritmo de Kruskal produce un árbol óptimo para todos los grafos con $n-1$ aristas.

Paso de inducción: Consideremos un grafo con n aristas. El algoritmo de Kruskal selecciona la arista de menor peso que no forma un ciclo. Esto da como resultado un subgrafo con $n-1$ aristas.

Por la hipótesis de inducción, el algoritmo de Kruskal produce un árbol óptimo para este subgrafo.

Cuando se agrega la n -ésima arista, se mantiene la propiedad del árbol porque la arista añadida es la de menor peso que no forma un ciclo. Por lo tanto, el algoritmo de Kruskal produce un árbol óptimo para el grafo con n aristas.

Por lo tanto, por el principio de inducción, el algoritmo de Kruskal produce un árbol óptimo para cualquier grafo.

4. Ejemplo paso a paso:

Calle entre la plaza 1 y la plaza 2 con un costo de 2.

Calle entre la plaza 1 y la plaza 3 con un costo de 3.

Calle entre la plaza 2 y la plaza 3 con un costo de 1.

Calle entre la plaza 2 y la plaza 4 con un costo de 3.

Calle entre la plaza 3 y la plaza 5 con un costo de 4.

Calle entre la plaza 4 y la plaza 5 con un costo de 2.

Fin del algoritmo: Todas las plazas están en el mismo conjunto, lo que significa que todas están conectadas por calles asfaltadas. Las calles seleccionadas forman el árbol de expansión mínima y el costo total es $1 + 2 + 2 + 3 = 8$.

El código implementado en C++ del algoritmo está en el fichero "5.cpp"